

Advanced C Course

Code Quality applied to C - Session 2

Pascal Rothnemer

10 February 2026, EPITA

Course Program - Sessions 1-7

1) Hello, World !

- C/C++ presentation and history
- Code quality : Compliance with Specs, Readability
- IDE setup

2) Types, objects, pointers

- **Developing on VS 2022**
- **Code quality : Useability, Documentation**

3) C++, OOP, Classes

- Classes, OOP
- Code quality : Useability, Documentation

4) Functions, libraries

5) Arrays, Vectors

- Debugging on VS 2022
- Code quality : Robustness, exception handling

6) Loops

- metrics on VS 2022
- Code quality : Performance, metrics , optimization

7) Projects Presentations & Evaluation

Exercise – VS Demo

- Write a program that Calculate square of input float **x**
- Use a function, two source files

Exercise

- Code a C program that calculate and display all prime numbers < 1000

Function « IsPrime » provided

```
bool isPrime(int n) {  
    if (n <= 1)  
        return false;  
  
    // Check divisibility from 2 to n-1  
    for (int i = 2; i < n; i++)  
    {  
        if (n % i == 0) return false;  
    }  
  
    return true;  
}
```

Main Code

```
int main()
{
    Int n = 0, nbprimebelow1000 = 0;
    while (n < 1000)
    {
        if (isPrime(n) == true)
        {
            std::cout << "\n prime integer found n = " << n << std::endl;
            nbprimebelow1000++;
        }
        n++;
    }
    std::cout << "\n there are " << nbprimebelow1000 << " prime numbers below 1000 \n";
}
```

Exercise

- Code a C program that increments a counter 1 billion times – AND measure the time elapsed
- Do the SAME in Python and measure time as well
- Compare results

Exercise

Hint : to calculate time elapsed :

```
using namespace std;
```

```
chrono::steady_clock::time_point begin = chrono::steady_clock::now();// before starting the loop
```

```
[...]
```

```
chrono::steady_clock::time_point end = chrono::steady_clock::now();// after finishing the loop
```

```
// calculate the difference and display it
```

```
cout << "Elapsed Time (sec) = " << (chrono::duration_cast<chrono::microseconds>(end - begin).count())  
/ 1000000.0 << std::endl;
```

Current State of C and C++

- **C23**, formally ISO/IEC 9899:2024, is the current open standard for the C programming language
 - Supersedes C17 (standard ISO/IEC 9899:2018).
 - Started in 2016 as C2x,[2], published on October 31, 2024
- **C++23**, formally ISO/IEC 14882:2024, is the current open standard for the C++ programming language
 - Follows C++98, C++03, C++11, C++14, C++17, C++20, C++2a
 - TBA : C++26

Can C++ Popularity today

- [Tiobe Index](#)
- **https://www.reddit.com/r/softwarearchitecture/comments/pt3xk2/c_vs_rust_vs_go/?rdt=54112**

Keywords

Reserved words, case sensitive : cannot be used for any purposes other than those for which they are predefined. 32 in C89. Most important ones:

auto break case char const continue default do
double else enum extern float for goto if int long
register return short signed sizeof static struct switch
typedef union unsigned void volatile while

Data Types

Type	Memory size	Values Range
char	1 byte	0 to 255
int	4 bytes	-2,147,483,648 to 2,147,483,647
float	4 bytes	1.2E-38 to 3.4E+38
double	2 or 4 bytes	2.3E-308 to 1.7E+308
bool	2 or 4 bytes	0 or 1
short	2 bytes	-32,768 to 32,767

Constant keyword

Prevents a variable from being modified.

=> need to initialize it at declaration time

```
const int n = 5;
```

```
n = 4; // IMPOSSIBLE – WILL NOT COMPILE
```

Keywords - Structure

Struct - defines a **composite, new, custom data type** - occupying a block of memory :

```
struct Point2D {
```

```
    float x;    // abscissa
```

```
    float y;    // ordinate
```

```
};
```

```
struct Point2D P = { 1.5, -2.0 };
```

Structures

Can contain various data types including structures.

```
struct point2D {
```

```
    float x; // abscissa
```

```
    float y; // ordinate
```

```
};
```

```
struct point3D {
```

```
    point2D p2D;
```

```
    float z; // z-axis coordinate
```

```
};
```

Structures

```
point2D My2DPoint;
```

```
My2DPoint.x = 2.0;
```

```
My2DPoint.y = -1.5;
```

```
point3D My3DPoint;
```

```
My3DPoint.p2d = My2DPoint;
```

```
My3DPoint.z = 0;
```

Unions

C unions are used to save memory. To better understand an union, think of it as a chunk of memory that is used to store variables of different types. When we want to assign a new value to a field, then the existing data is replaced with new data.

DON'T USE UNION IF NOT TO SAVE MEMORY (TYPICALLY EMBEDDED SOFTWARE WITH LIMITED RESOURCES)

Keywords - Union

Union - like struct, except that each data member is located at the same memory address.

Primary use : allowing access to a common location by different data types

```
union {
```

```
    float f;
```

```
    uint32_t d; // Assumes float is 32 bits wide
```

```
};
```

```
f = 3.14f;
```

```
std::cout << "Hexadecimal representation of 3.14f:" << std::hex << d << '\n';
```

Typedef

- Creates an alias for a data type
- does not create a new type
- Often used to simplify syntax of declaring complex data structures

`typedef int length;` // creates type length synonym of int

`length DistInMeters = 1;` // use it

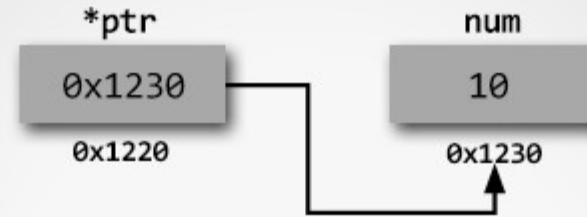
Pointers

- A particular **type** of variable that
 - Stores the address of an object
 - Serves to « **access** » that object.
- A Pointer « points » to a **typed** memory address
 - `int* Ptr; // defines a pointer to an int`
 - `float* Ptrf; // defines pointer to a float`

Pointers

A variable that stores a memory address.

- efficient handling of **arrays** and **structures**.
- Useful to pass & return **multiple values** from a function.
- allow dynamic memory allocation

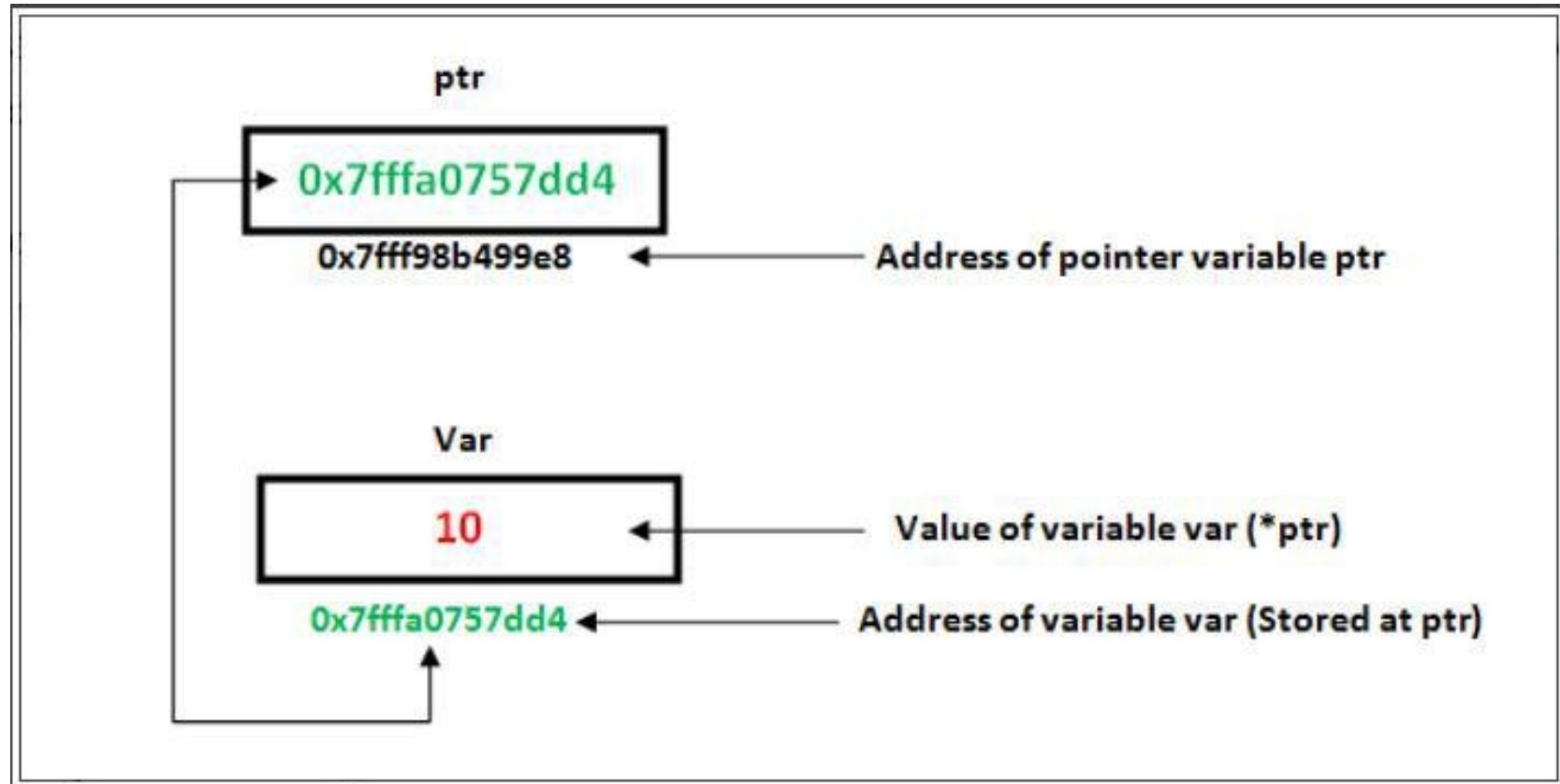


Pointer to integer



Pointer to pointer

Pointers



« * » Symbol

means « Pointer to » (type)

`int n ;` // defines an integer n

`int* ptr;` // defines a pointer to an int

« & » Symbol : « Address of »

Unary operator - returns the memory address of its operand.

```
int a = 7, b;
```

```
int* p; // Un-initialized Pointer
```

```
p = &a; // Stores « address of a » in p
```

```
b = *p + 1; // value of b ?
```

Pointer Initialization

```
int* p3 = &N ;
```

// IS EQUIVALENT TO :

```
int* p3 ;
```

```
p3 = &N ;
```

Pointer to Pointer - Double Pointer

```
int a = 7, b, c, d, e;
```

```
int* p;
```

```
p = &a;
```

```
b = *p + 1;
```

```
int** pp = &p;
```

```
c = ++(**pp); // value of c?
```

```
d = (**pp)++; // value of d?
```

```
e = ++(**pp); // value of e?
```



Pointer to array - syntax

```
int array[] = { 9,8 };
```

```
int* j = array;
```

```
// EQUIVALENT TO :
```

```
int array[2] ;
```

```
array[0] = 9 ;
```

```
array[1] = 8 ;
```

```
int* p;
```

```
p= &array ; // or j = &array[0]
```

Pointers exercise

```
int a = 7, b, c, d, e;
```

```
int* p; // Un-initialized Pointer
```

```
p = &a; // Stores address of a in p
```

```
b = *p + 1; // value of b ?
```

Values of a, b, c, d, e ?

```
int** pp = &p;
```

```
c = ++(**pp); // value of c?
```

```
d = (**pp)++; // value of d?
```

```
e = ++(**pp); // value of e?
```

Pointer to a Structure

```
struct point {
```

```
    float x;
```

```
    float y;
```

```
};
```

```
point P;
```

```
P.x = 0.0;
```

```
P.y = -2.0;
```

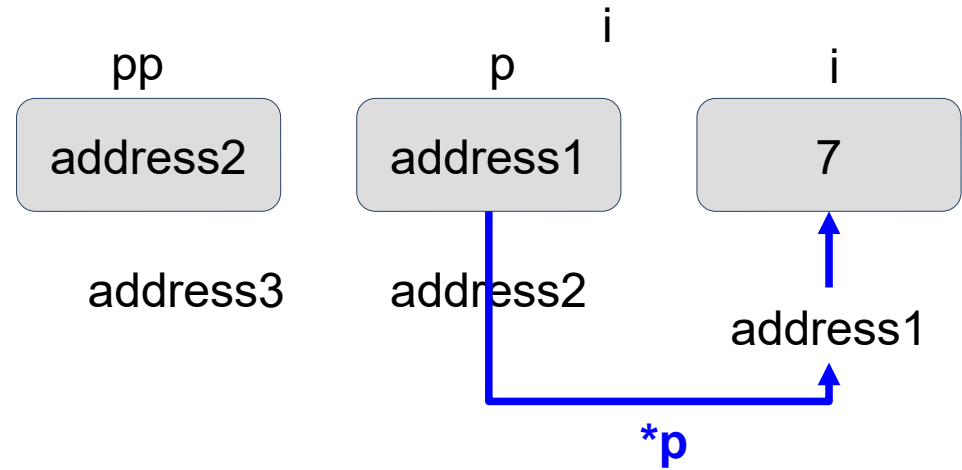
```
point* ptr = &P;
```

```
ptr→x = 1.0;
```

```
ptr→y = -4.0;
```

Pointers exercise

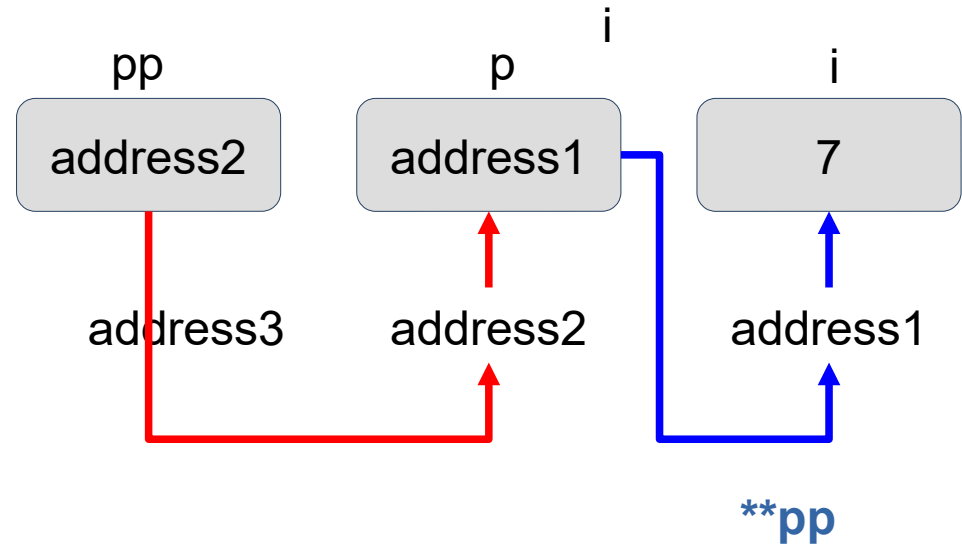
$*p$ is the value pointed by
p : i.e. content of address
contained by p, i.e. content
of **address1**, i.e. **7**



Pointers exercise

a double pointer is a variable that points to another pointer which in turn, points to an object in memory.

****pp** is the value pointed by the pointer pointed by pp : content of the address pointed by **address2** of p : i.e. content of address1, i.e. **7**



Pointers exercise

```
int i = 7, j;
```

```
int* p = &i;    // creates ptr p, Stores address of i in p
```

```
int** pp = &p; // creates dble ptr pp, stores address of p in pp
```

```
j = ++(**pp);  // FIRST increment(**pp) => i = 8
```

```
                // THEN assign value **pp to j => j = 8
```

```
i++;           // Increment i by one => i = 9
```

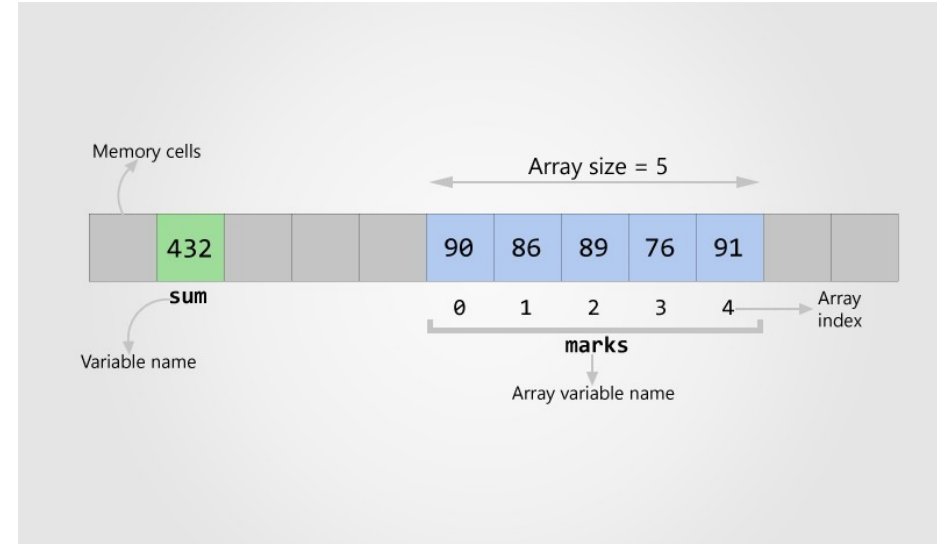
```
j = (**pp)++;  // FIRST assign value **pp to j => j = 9
```

```
                // THEN increment(**pp) => i = 10
```

Arrays

Data structure holding **finite sequential** collection of **homogeneous** data.

- **finite** – nb of data is finite, fixed prior to use.
- **sequential** in memory → can be indexed.
- **homogeneous** – The data are of the same type.
- **One-dimensional** or **Multi-dimensional** array



Arrays

```
int marks[5]; // defines a one-dimensional array of 5 integers  
  
for (int i = 0 ; i < 5 ; i++)  
{  
    marks[i] = 0 ; // init array via a loop over array index  
}
```

Arrays

`int marks[5];` // defines a one-dimensional array of 5 integers

`for (int i =0 ; i<=5 ; i++)`

`{`

`marks[i] = 0 ;` // init array via a loop over array index

`}`

Arrays

```
int marks[5]; // this defines an array of 5 integers

// the line below defines and initializes an array of 5 integers

int marks[5] = { 90, 86, 89, 76, 91 };

// the lines below initialize the array element by element

marks[0] = 90; // Assigns 90 to first element of marks array
marks[1] = 86; // Assigns 86 to second element of marks array
marks[2] = 89; // Assigns 86 to second element of marks array
marks[3] = 76; // Assigns 86 to second element of marks array
marks[4] = 91; // Assigns 91 to fifth element of marks array
```

Arrays – Dynamic Memory Allocation

`int marks[10];` // array length is 10. What if we need to change it

A runtime case needs only 6 elements → 4 indices are wasting memory. We need to decrease length from 10 to 5.

If on contrary there is a need for 3 more elements, 3 indices more are required. The length needs to increase from 10 to 13.

This is possible thanks to **Dynamic Memory Allocation** in C : the size of an Array can be changed at runtime. 4 functions provided by C under `<stdlib.h>` are :

Malloc(), calloc(), free(), realloc()

Arrays – Dynamic Memory Allocation

```
ptr = (int*) malloc(100 * sizeof(int)); //allocates 100 int
```

```
ptr = (float*) calloc(25, sizeof(float)); //contiguous alloc
```

```
free(ptr); // de-allocates the memory.
```

```
ptr = realloc(ptr, newSize); // changes memory allocation
```

Arrays – exercise

write a Program that

- defines an array of 11 floats
- Let user enter 11 float values
- Calculate the average of the values, display it
- optional : Calculate the median of an array of 11 values, display it

TIP : sort array the $(11+1)/2 = 6$ th element of this sorted array is the median of the original array

Arrays – sorting by ascending order

```
int i,j,c;  
for(i = 0; i < N-1; i++)  
    for(j=i+1;j<N;j++)  
        if ( T[i] > T[j] ) {  
            c = T[i];  
            T[i] = T[j];  
            T[j] = c;  
        }
```

Find the minimum of the array and save it in 1st position.

Once done do the same for the remaining part of the array by positioning the new minimum at the second location, and so forth...

At the end of this double loop, the array is sorted by ascending order

Session 2 – Code Quality Overview

- C/C++ fundamentals – contd
- **Code Quality Overview**

Overview - What is Code Quality



Session 2

«Code is like humor. If you have to explain it, it's bad ».

(Cory House)

«Code is read more often than it is written».

(Guido van Rossum, creator of Python)

Session 2

«“It’s harder to read code than to write it.” ».

«If, at first, you do not succeed, call it version 1.0».

Software activities

What does it take to have (good) software available at the end of a project ?



Software activities

What does it take
to have (good)
software
available at the
end of a project ?

- Read/write requirements
- Write specifications
- System Architecture (Define sub-systems, Components and their interactions)
- Design code components
- Code
- Test
- Document
- Deploy
- Etc...

Quality – Requirements & Specifications

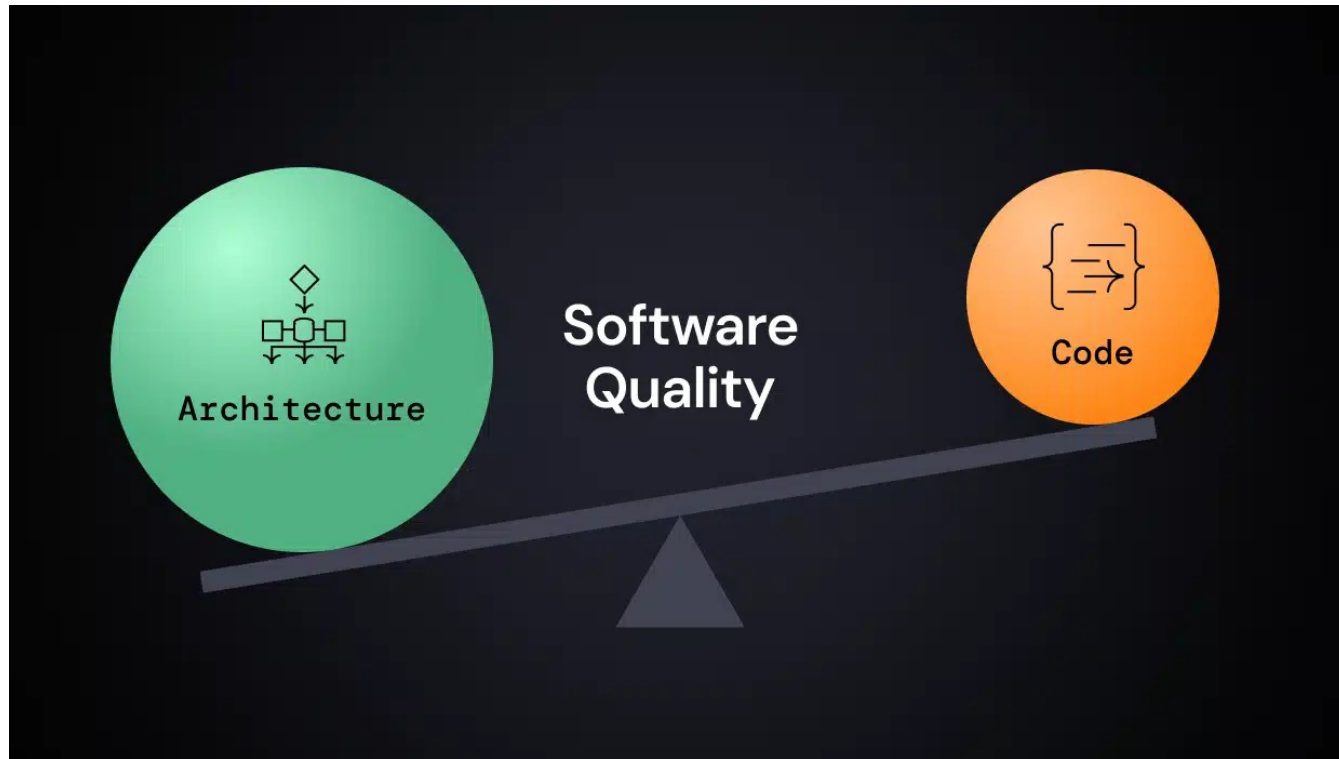
Describe **WHAT** the system will do:

- **functional** and **non-functional** requirements
- may include a set of **use cases** that describing **system/user interactions**



Quality – Architecture

Set of structures needed to reason about a **software system** and **the discipline of creating such structures and systems**. Each structure comprises **software elements, relations among them**, and properties of both elements and relations.



Quality – Software Design

the process of conceptualizing **how** a software system will work before it is implemented or modified.

- the process of planning and structuring a software system before actual coding begins.
- Mainly : definition of data, classes, functions, and methods



Code Quality – Code writing

Writing good
code :

Code Quality – Definition attempt

- software **meets its current requirements and satisfies its current user needs** : « it's easy if you try ». Further, and assuming no software is future-proof—if it were, why wasn't it released sooner?— software quality can be defined by how easily it adapts:
- **Functionality & usability**: Can the software be easily modified, updated, and repaired to meet new requirements and usage patterns?
- **Security & portability**: Can updates to security vulnerabilities be made quickly without risking the stability of the software? Can the software run in new environments and platforms with minimal changes?
- **Reliability & quality**: Can the software perform reliably despite constant changes, with little impact on other components? Will it deploy efficiently with changes?

Code Quality - User

- Code's **effectiveness, reliability, security**.
- how well **it runs as intended/documentated**.
- **performant** and **efficient**, **runs seamlessly** and **responds quickly**.
- **easy to extend** to add **new features** and **functionalities**.

Code Quality - Developer

- Easy to **read, clear** and **modular...**
- Easy to **understand, change, operate, debug** and **maintain.**
- Comply with **established coding standards & practices**
- easy to **extend** to new **features/Functionalities.**

Why Code Quality is Important

- Compliance with the specification : contract between developer and user, i.e. software does what it is supposed to do
- Optimizes software product cost during its whole lifecycle : improves code readability, maintainability, effectiveness, and reusability. Reduces issues such as code smells, bugs, security vulnerabilities, and the accumulation of code-level technical debt.

Practices Supporting Good Code Quality

- **Clear requirements & design:** Clear and precise requirements and a detailed blueprint of the potential dependent elements ensure that engineers understand the task at hand. Bad requirements and design will lead to confusion and possible rework.
- **Robust testing & QA:** Conducting various levels of testing (unit, integration, system, load, and acceptance testing) ensure the software functions correctly and meets quality standards. The more automated and complete the testing, the easier it is to minimize the time to release software. Less dependencies for the changed element allow QA to focus on the specific functionality rather than conducting acceptance on the entire system.
- **Automated CI/CD, security testing and tooling:** Automating the process of code integration, testing, vulnerability testing, and deployment to ensure rapid and reliable delivery of software updates.
- With these elements, software engineers can develop high-quality software that meets user expectations and performs reliably. However, while these practices support a good software development lifecycle, they do not ensure that the software itself is of good quality.

Code Smells - Definition

Code Smells are traces in the code that indicate a deeper problem in the application or codebase. They are not bugs or errors but are tangible and observable violations of code design/development fundamentals that could eventually lead to poor code quality and technical debt.

Code Smells – « Magic Numbers »

```
struct student {  
    string name;  
    string cursus;  
    int age;  
};  
  
// Array of Students to store data i  
student stu[200], tempstu[200],  
sortstu[200], sortstu1[200];
```

```
#define maxStuds 200  
struct student {  
    string name;  
    string cursus;  
    int age;  
};  
  
// Array of Students to store data i  
student stu[maxStuds],  
tempstu[maxStuds],  
sortstu[maxStuds],  
sortstu1[maxStuds];
```

Code Smells - Examples

- large classes, long methods or functions
- one function trying to do too much, does more than one thing → deserves having a look, possible refactoring & split into two smaller ones

Code Smells – Confusing Variables & Functions Names

- « A long descriptive name is better than a short enigmatic name. »
- « A long descriptive name is better than a long descriptive comment. »

Code Smells – Duplicated Code

```
extern int array_a[];
extern int array_b[];
int sum_a = 0;

for (int i = 0; i < 4; i++)
    sum_a += array_a[i];

int average_a = sum_a / 4;
int sum_b = 0;

for (int i = 0; i < 4; i++)
    sum_b += array_b[i];

int average_b = sum_b / 4;
```

```
int calc_average_of_four(int* array) {
    int sum = 0;
    for (int i = 0; i < 4; i++)
        sum += array[i];
    return sum / 4;
}

extern int array1[];
extern int array2[];

int average1 = calc_average_of_four(array1);
int average2 = calc_average_of_four(array2);
```

Technical Debt - Definition

It is the **cost of future reworking** because **prioritizing speedy delivery** over **good quality code**. Such debt can impact system performance, scalability or resilience.

Like monetary debt, if technical debt is not repaid, it can accumulate "interest", making it harder to implement changes.

Technical Debt - Examples

- Use of Legacy code
- Insufficient testing
- Hard-coded values
- Outdated libraries or dependencies
- Lack of documentation or outdated one
- Suboptimal architecture
- Inconsistent coding standards
- Deferred upgrades

Maintainability

- High-quality code is easier to read, understand, and modify.
- Clear code enhances communication between team members is enhanced when Same for code adhering to uniform conventions and standards.
- Improves maintainability, making it easier for developers to address faults, and add new features

Reliability & Robustness

- manage exceptions and issues to keep the program reliable under difficult situations.
- Manage « Unhappy » PATHS – not only the HAPPY PATH
- Software that is less susceptible to crashes, data loss or corruption, and vulnerabilities.

Why Code readability in C is important

C is a powerful programming language, low-level, allowing Complex Code, and has a steep learning curve. It can be challenging to read and understand. And it is often used in Large Projects → Here are some of the reasons why code readability is essential when writing C:

•Easy to Understand

When code is readable, it is easier for other programmers to understand how the code works. This can be helpful when working with code that has been written by someone else or when returning to a project after an extended period.

2. Easier to Debug

When code is readable, it is easier to debug. Debugging is the process of finding and fixing errors in code. When code is easy to read, it is easier to identify where errors are located and what is causing them.

3. Better Collaboration

Readability also helps with collaboration. When code is easy to read, it is easier for other developers to work with the code. This can be helpful when working on large projects with multiple developers. Readable code also promotes better communication among developers, leading to a more efficient workflow.

4. Easier Maintenance

Finally, readability is critical for code maintenance. When code is easy to read, it is easier to maintain. Maintenance is the process of updating and modifying code. When code is readable, it is easier to make changes and add new features to the codebase.

How to improve Code Quality

- Throughout the software development lifecycle, developers can use a number of methods and techniques to enhance the quality of their code.
- First and foremost, it's crucial to write readable, consistent code.
- Adopting sound coding habits, leveraging the right tools, and adhering to set norms are all necessary steps in the continuous process of improving code quality.
- Following coding conventions, using best practices, performing extensive testing, adopting code reviews, utilizing automated tools, integrating CI/CD pipelines, and cultivating a culture of continuous improvement is a couple of ways to help enhance code quality.

Tips for writing Readable Code

- Use Meaningful Names

```
int d; // days until deadline  
int daysUntilDeadline;
```

- Use known abbreviations
- Prefer clarity over brevity
- Use accepted conventions
- Name Functions/Method start with a verb

Use Consistent
Formatting

Avoid Complex Logic

Use Comments

Follow Best Practices

Testing

- In-depth testing is essential for enhancing code quality.
- To verify the accuracy and functionality of the code, developers must use unit tests, integration tests, and other automated testing approaches.
- It is possible to use test-driven development (TDD), in which tests are written before the actual code.
- This method makes sure the code complies with the requirements and assists in finding and fixing the problems early in the development process.

Code Reviews

- Can be an effective way to improve code quality, allows the developer to assess the quality assurance of the codebase.
- Peer or manual code review lets programmers give feedback, spot potential problems, and exchange expertise. Typically seen as a more collaborative process within the same team or organization.
- Static code analysis tools scan the codebase for possible issues like syntax errors, code smells, performance bottlenecks or security vulnerabilities.
- These technologies can be incorporated into the development workflow to help developers spot and fix code quality problems early, which lowers the risk of writing inefficient or bug-ridden code.

Code Analysis Tools

Static Code Analysis

- Identifies potential bugs, security vulnerabilities, style issues and design issues (abstraction levels for instance)
- Example Visual Studio (depending on level), SonarQube...

Dynamic Code Analysis

- Analyzes code while it is running. It can measure the **performance, behavior**, and functionality of the code, and identify runtime errors, memory leaks, resource consumption, and other issues that may affect the user experience.
- pin is the bottleneck